

# **Sentinel: Adversarial Fraud Detection AI**

## **Comprehensive Project Documentation**

**By:**

**Nick Mwaura.**

**&**

**Festus Koech.**

**Final Project**

**MSc. Computer Science**

***CSCI 5651: Data Science***

***Professor Kamala Marepalli***

## Problem Statement & Data Science Context

Credit-card fraud is a binary classification problem under extreme class imbalance: fewer than 1% of transactions are fraudulent. Fraudsters adapt over time (adversarial drift), so a production system must not only train an accurate classifier but also monitor whether incoming data still resembles the training distribution. This project applies the full data science lifecycle: problem definition → EDA → ETL → modelling → evaluation → deployment → monitoring.

- Business problem: flag fraudulent transactions with high recall while limiting false alarms.
- Data problem: ~1000:1 class imbalance; labels only available historically.
- Technical problem: raw CSV contains PII, mixed types, and geographic fields requiring engineering.
- Operational problem: model must be served via API/UI with drift governance.

### End-to-End Data Science Workflow Used

| Phase                  | What we did                                              | Project artifact                |
|------------------------|----------------------------------------------------------|---------------------------------|
| 1. Problem framing     | Defined fraud detection + drift monitoring goals         | README, config.yaml             |
| 2. EDA                 | Explored imbalance, amounts, categories, time, geography | Section 3, eda_plots/           |
| 3. ETL                 | Featurize, clean, SMOTE train, hold-out test             | src/etl/, src/data/credit_dt.py |
| 4. Feature engineering | StandardScaler, PCA viz                                  | src/engineering/, scaler.joblib |
| 5. Modelling           | CatBoost + Isolation Forest                              | catboost_fraud.cbm              |
| 6. Evaluation          | ROC-AUC, F1, confusion matrix                            | model_metrics_snapshot.json     |
| 7. Deployment          | FastAPI + Sentinel UI                                    | src/api/, frontend/             |
| 8. Monitoring          | KS drift + governance                                    | drift_report.csv, policy.py     |

## 1. Abstract

**Purpose:** We developed Sentinel, an end-to-end financial fraud detection application that scores credit-card transactions in batch and near-real-time, monitors feature drift, and applies business governance when data distributions shift.

**Primary algorithm:** CatBoost (Categorical Boosting) gradient boosted decision trees — 500 trees, maximum depth 6, learning rate 0.05, with `auto_class_weights=Balanced`.

**Supporting algorithms:** SMOTE for training-time class balancing; StandardScaler for feature normalization; Isolation Forest for unsupervised anomaly detection; Kolmogorov–Smirnov (KS) tests for distributional drift monitoring.

**Data source:** publicly available `credit_dt` dataset — `fraudTrain.csv` (~1.3 million rows) and `fraudTest.csv` (~555,000 rows) containing transaction amount, merchant category, cardholder and merchant coordinates, timestamps, gender, and US state. Training used 200,000 train rows and 100,000 test rows (`config.yaml` caps).

Results on held-out `fraudTest.csv`:

| Metric            | Value  |
|-------------------|--------|
| ROC-AUC           | 0.9851 |
| F1 Score          | 0.6887 |
| Precision         | 0.7039 |
| Recall            | 0.6741 |
| Average Precision | 0.7093 |
| Balanced Accuracy | 0.8365 |

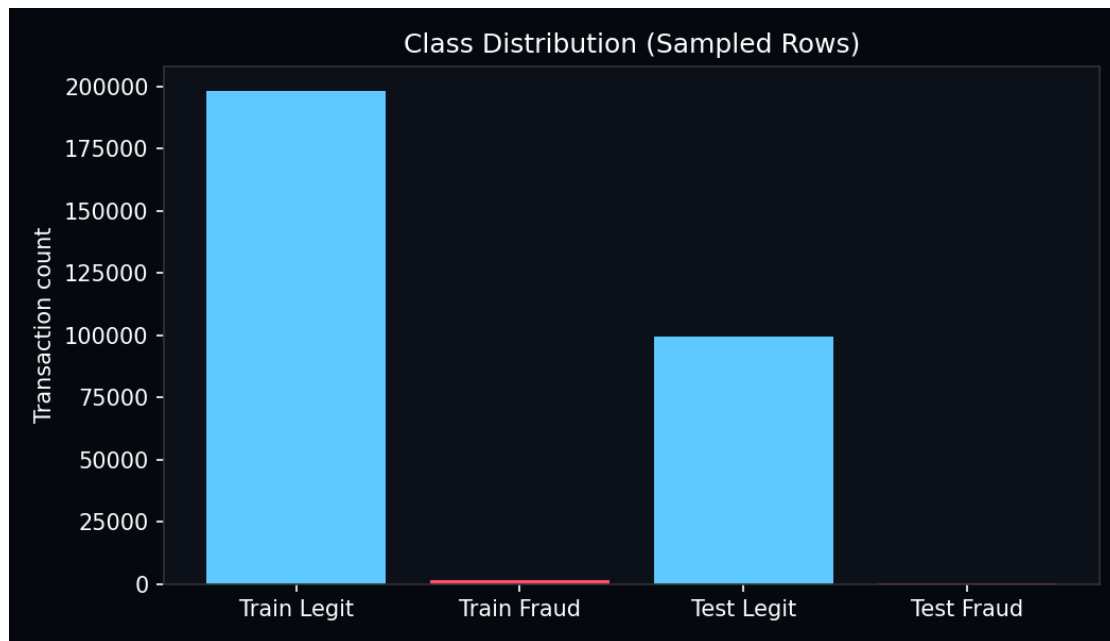
## 2. Exploratory Data Analysis (EDA) → ETL Pipeline

Before modelling, we explored raw fraudTrain/fraudTest CSVs to understand data quality, class balance, and feature behaviour. Findings directly informed ETL and feature engineering.

### 2.1 EDA Findings

- Class imbalance: ~0.82% fraud in train sample, ~0.40% in test — SMOTE required on train.
- Amount skew: fraud mean ~\$509 vs legitimate ~\$68 — amt is a strong predictor.
- Category: 14 merchant types with varying fraud rates — top CatBoost feature (~64% importance).
- Time: fraud rate varies by hour — trans\_hour and trans\_dow engineered from timestamp.
- Geography: cardholder vs merchant coordinates motivate Haversine distance\_km feature.
- No missing values in sampled data; duplicates removed in ETL clean step.

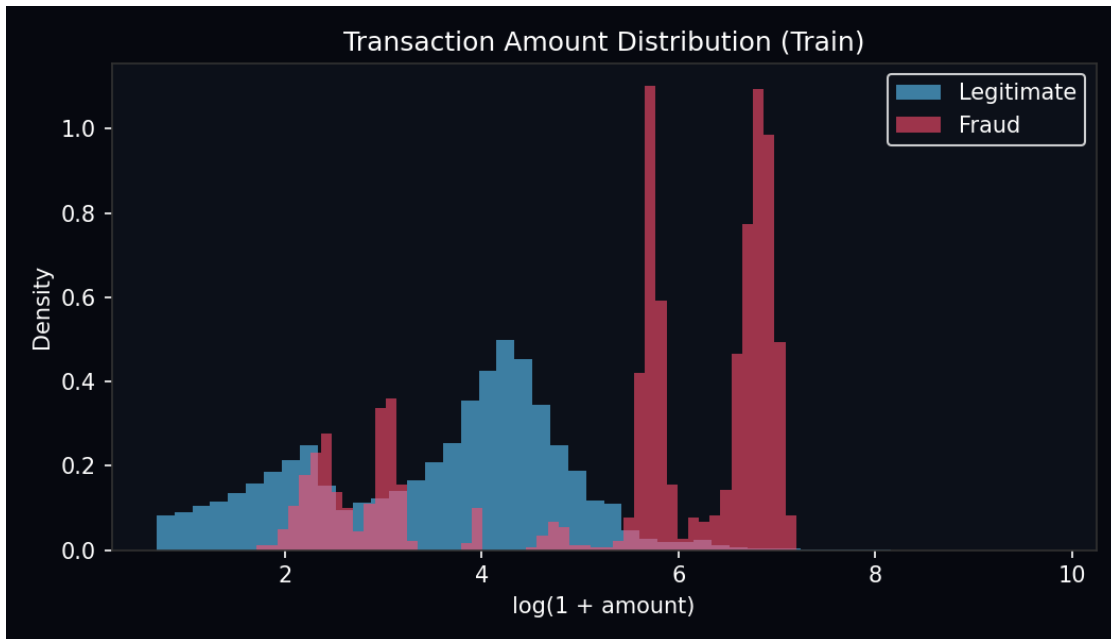
**Figure 1: Class distribution (train vs test sample)**



Interpretation:

- Blue bars represent legitimate transactions; red bars represent known fraud.
- The severe imbalance (~1000:1) explains why accuracy alone misleading and SMOTE is applied.
- Test fraud rate is lower than train — evaluation must use precision/recall and ROC-AUC.

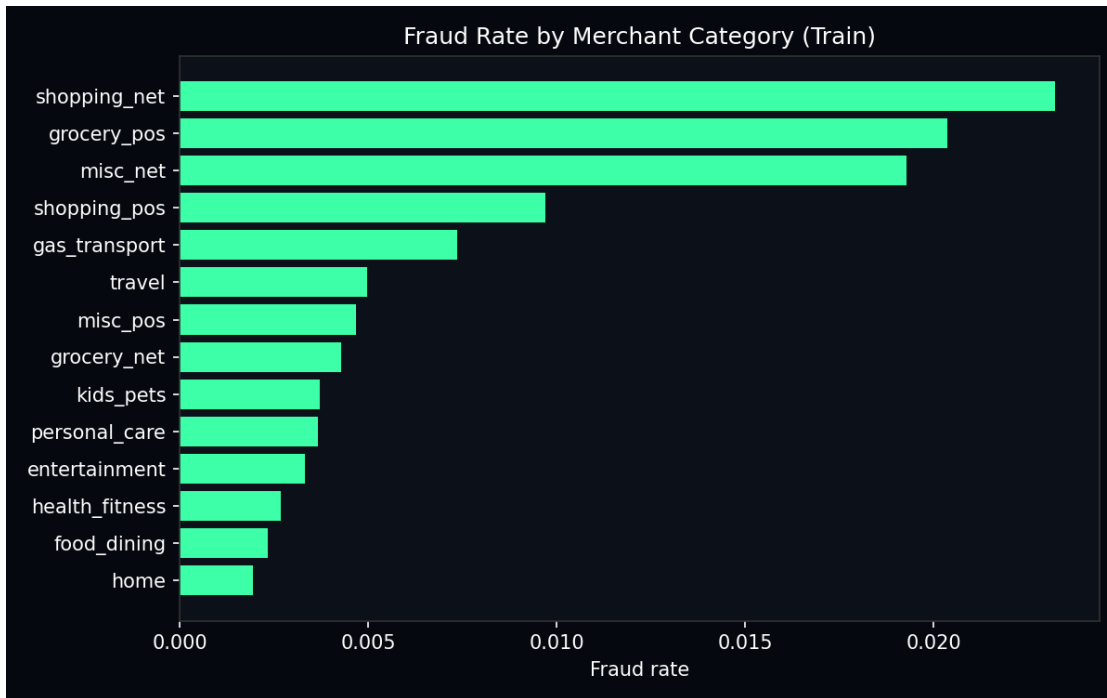
**Figure 2: Log amount distribution by class**



Interpretation:

- Histogram of  $\log(1+\text{amount})$  separates fraud (red) and legitimate (blue) distributions.
- Fraudulent purchases tend toward higher dollar amounts — supports using amt as a numeric feature.
- Log transform visualizes skew; model uses raw amt after StandardScaler.

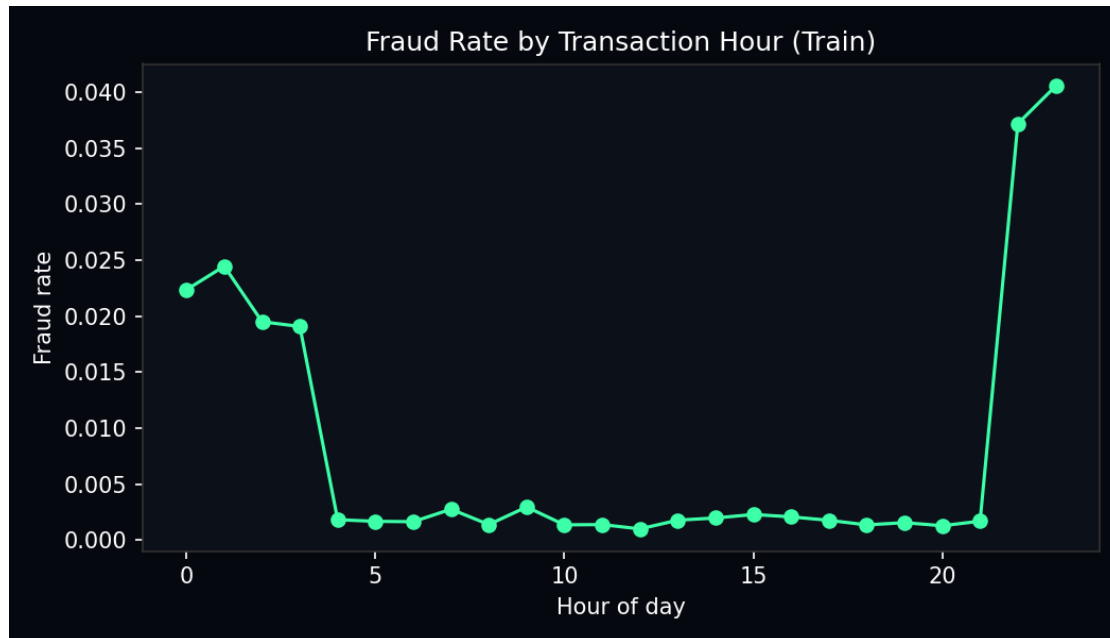
**Figure 3: Fraud rate by merchant category**



Interpretation:

- Each bar is a merchant category (e.g. shopping\_net, travel, grocery).
- Wide variation in fraud rate across categories justifies label-encoding category for CatBoost.
- EDA confirmed category would dominate feature importance — validated post-training at 64%.

**Figure 4: Fraud rate by hour of day**



Interpretation:

- Line chart shows how fraud probability changes across 24 hours.
- Temporal patterns justify extracting `trans_hour` and `trans_dow` from `trans_date_trans_time`.
- Later KS drift on `unix_time/trans_dow` reflects temporal shift between train and test splits.

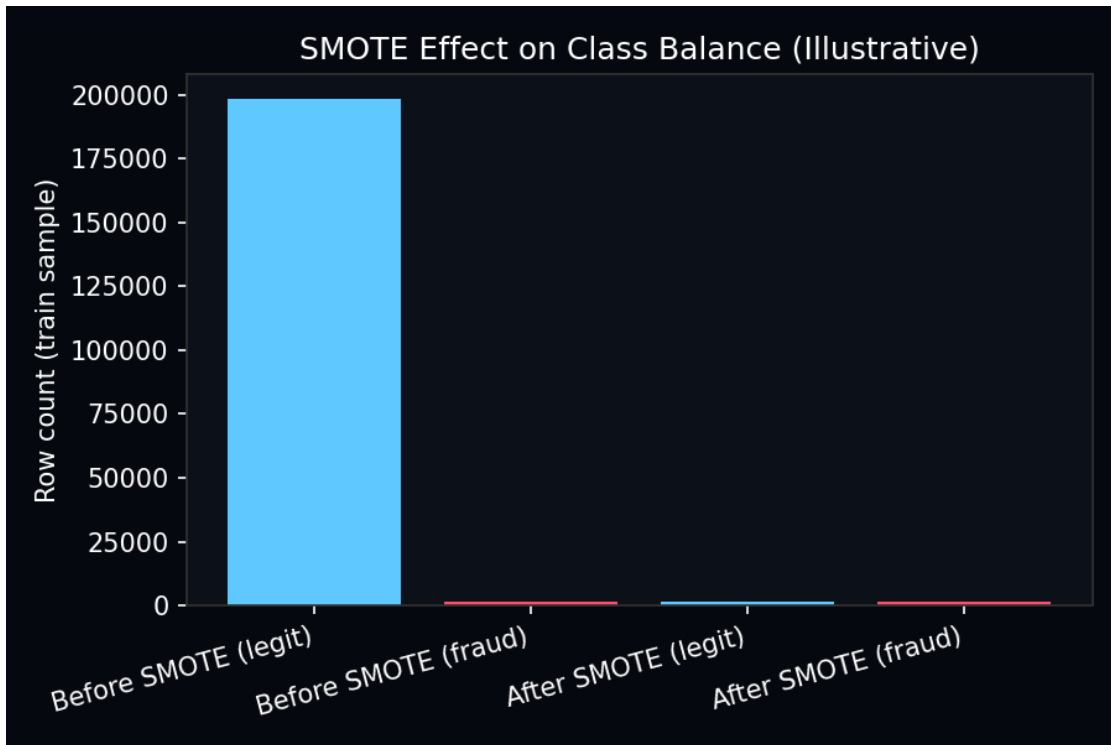
## 2.2 ETL Process (Transform after EDA)

ETL stages implemented in `src/etl/pipeline.py` and `src/data/credit_dt.py`:

| Step                  | Operation                                                     | Rationale from EDA                      |
|-----------------------|---------------------------------------------------------------|-----------------------------------------|
| Extract               | Load <code>fraudTrain.csv</code> / <code>fraudTest.csv</code> | Provider pre-split; no random split     |
| Transform — featurize | Haversine distance, time fields, label encoding               | EDA showed geo + time + category matter |
| Transform — clean     | Drop duplicates, median impute                                | Data quality assurance                  |
| Transform — SMOTE     | Balance train labels only                                     | EDA confirmed <1% fraud rate            |

|      |                                                 |                                        |
|------|-------------------------------------------------|----------------------------------------|
| Load | X_train, X_test, y_train,<br>y_test + artifacts | Ready for StandardScaler +<br>CatBoost |
|      |                                                 |                                        |

**Figure 5: SMOTE effect on class balance**



Interpretation:

- Before SMOTE: ~198k legitimate vs ~1.6k fraud in train sample.
- After SMOTE: classes balanced via synthetic fraud points interpolated from k=5 neighbours.
- Test set is never SMOTE'd — evaluation reflects real-world fraud rate (~0.4%).



### 3. Design — Modules & Architecture

Sentinel follows a seven-layer AI World architecture. Upper layers decide policy; lower layers compute predictions and statistics.

#### Seven-Layer AI World Architecture

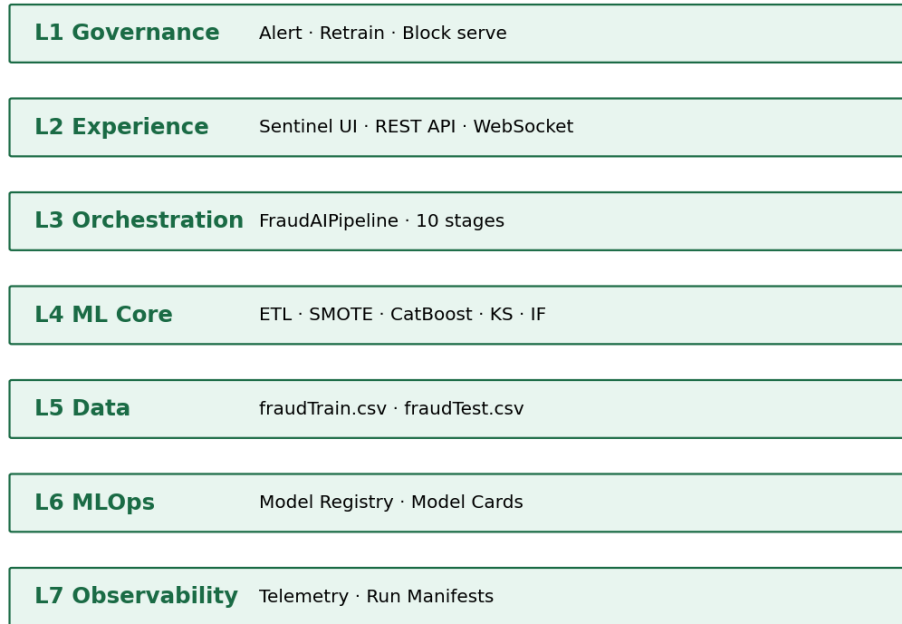


Figure 6: Architecture diagram — data flows from L5 (CSV) through L4 (ML) to L2 (API/UI), with L1 governance reacting to drift and L7 logging every pipeline run.

#### 3.1 Module Details

| Layer            | Module path                                     | Responsibility                                   |
|------------------|-------------------------------------------------|--------------------------------------------------|
| L1 Governance    | src/governance/policy.py                        | Map drift alerts + ROC-AUC → alert/retrain/block |
| L2 Experience    | src/api/, frontend/, dashboard/                 | Sentinel UI, REST, WebSocket, Streamlit          |
| L3 Orchestration | src/orchestration/pipeline.py                   | FraudAIPipeline stage execution                  |
| L4 ML Core       | src/etl/, engineering/, inference/, monitoring/ | ETL, SMOTE, CatBoost, KS, IF                     |
| L5 Data          | src/data/credit_dt.py                           | CSV load, featurizer, stable encodings           |
| L6 MLOps         | src/registry/model_registry.py                  | Versioning, production promotion, model cards    |
| L7 Observability | src/observability/telemetry.py                  | JSONL events, run manifests                      |

### 3.2 Pipeline Stages (Orchestration)

- DATA\_INGEST → validate CSV paths
- ETL → featurize, clean, SMOTE (train only)
- FEATURE\_ENGINEERING → StandardScaler + PCA plot
- TRAIN → CatBoost fit
- EVALUATE → ROC-AUC, F1, confusion matrix
- DRIFT\_BASELINE → save train feature distributions
- DRIFT\_CHECK → KS test train vs test
- ANOMALY\_MODEL → Isolation Forest
- REGISTER → promote to production registry

## 4. Algorithms & Sample Code

This section presents the mathematical foundations and annotated Python implementations used in the project.

### 4.1 Haversine Distance (Feature Engineering)

**src/data/credit\_dt.py**

```
def _haversine_km(lat1, lon1, lat2, lon2):
    r = 6371.0
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    return 2 * r * np.arcsin(np.sqrt(np.clip(a, 0, 1)))
```

| Code line / block                              | Explanation                                                        |
|------------------------------------------------|--------------------------------------------------------------------|
| <code>r = 6371.0</code>                        | Earth radius in kilometres for great-circle distance.              |
| <code>map(np.radians, ...)</code>              | Convert degrees to radians — required by trig functions.           |
| <code>dlat, dlon</code>                        | Difference in latitude/longitude between cardholder and merchant.  |
| <code>a = sin<sup>2</sup>(Δlat/2) + ...</code> | Haversine formula intermediate term.                               |
| <code>2 * r * arcsin(√a)</code>                | Final distance in km — stored as <code>distance_km</code> feature. |

### 4.2 ETL Pipeline (SMOTE + Clean)

**src/etl/pipeline.py**

```
def clean(self, df):
    out = df.copy()
    if self.etl_cfg.get("drop_duplicates", True):
        out = out.drop_duplicates()
    for col in out.select_dtypes(include=[np.number]).columns:
        if out[col].isna().any():
            out[col] = out[col].fillna(out[col].median())
    return out

def apply_smote(self, X, y):
    smote = SMOTE(sampling_strategy="auto", k_neighbors=5, random_state=42)
    X_res, y_res = smote.fit_resample(X, y)
    return pd.DataFrame(X_res, columns=X.columns), pd.Series(y_res, name=y.name)
```

| Code line / block | Explanation |
|-------------------|-------------|
|-------------------|-------------|

|                                 |                                                                        |
|---------------------------------|------------------------------------------------------------------------|
| <code>df.copy()</code>          | Work on a copy — avoid mutating original dataframe.                    |
| <code>drop_duplicates()</code>  | Remove exact duplicate rows found during EDA.                          |
| <code>fillna(median)</code>     | Impute missing numeric values with column median.                      |
| <code>SMOTE(...)</code>         | Create synthetic fraud samples between $k=5$ nearest fraud neighbours. |
| <code>fit_resample(X, y)</code> | Return balanced feature matrix and labels for training only.           |

### 4.3 CatBoost Classifier

`src/inference/engine.py`

```
model = CatBoostClassifier(
    iterations=500, learning_rate=0.05, depth=6,
    eval_metric="AUC", auto_class_weights="Balanced", random_seed=42)
model.fit(X_train_scaled, y_train)
proba = model.predict_proba(X_test)[:, 1]
preds = (proba >= 0.5).astype(int)
```

| Code line / block                        | Explanation                                                         |
|------------------------------------------|---------------------------------------------------------------------|
| <code>iterations=500</code>              | Build 500 sequential decision trees (boosting rounds).              |
| <code>depth=6</code>                     | Each tree asks up to 6 split questions — controls model complexity. |
| <code>learning_rate=0.05</code>          | Shrinkage — each tree contributes 5% weight to ensemble.            |
| <code>auto_class_weights=Balanced</code> | Penalise misclassified fraud more heavily.                          |
| <code>predict_proba[:, 1]</code>         | Return $P(\text{fraud})$ — probability of positive class.           |
| <code>proba &gt;= 0.5</code>             | Default threshold: flag as fraud if probability $\geq 50\%$ .       |

### 4.4 StandardScaler

`src/engineering/features.py`

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

| Code line / block                   | Explanation                                                            |
|-------------------------------------|------------------------------------------------------------------------|
| <code>fit_transform(X_train)</code> | Learn $\mu$ and $\sigma$ from train; transform train to mean 0, std 1. |
| <code>transform(X_test)</code>      | Apply same $\mu$ , $\sigma$ to test — prevents data leakage.           |

#### 4.5 KS Drift Monitor

**src/monitoring/drift.py**

```
stat, p_value = ks_2samp(ref_train, cur_batch)
drift_detected = p_value < 0.05
```

| Code line / block | Explanation                                                  |
|-------------------|--------------------------------------------------------------|
| ks_2samp          | Two-sample Kolmogorov–Smirnov test per feature.              |
| ref_train         | Baseline distribution from pre-SMOTE training data.          |
| p_value < 0.05    | Reject null hypothesis — distributions differ significantly. |

#### 4.6 Full Training Entry Point

**main.py / src/orchestration/pipeline.py**

```
# Run complete pipeline:
python main.py
```

```
# Internally:
pipeline = FraudAIPipeline(load_config())
pipeline.run_training()
```

| Code line / block | Explanation                                                         |
|-------------------|---------------------------------------------------------------------|
| FraudAIPipeline   | Orchestrator executing all stages in order with shared context.     |
| run_training()    | Runs ingest → ETL → engineer → train → evaluate → drift → register. |

## Appendix A — Line-by-Line: Raw CSV Featurization

### src/data/credit\_dt.py — simplified

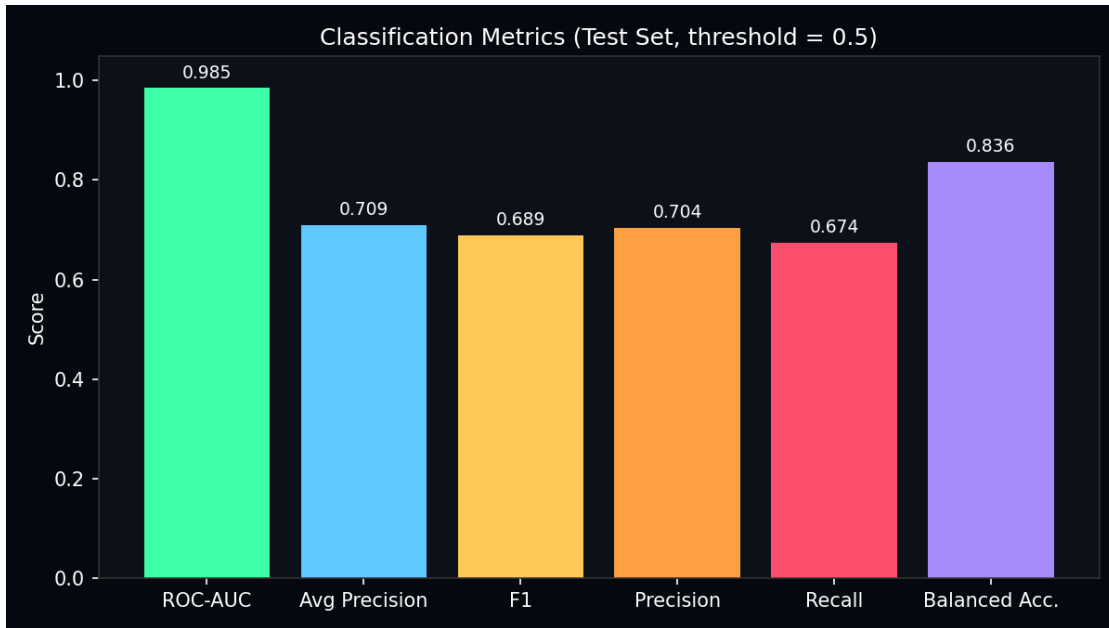
```
def transform_raw(self, raw_df):
    out = raw_df.copy()
    out["Class"] = out["is_fraud"].astype(int)      # Line 1: target
    ts = pd.to_datetime(out["trans_date_trans_time"]) # Line 2: parse time
    out["trans_hour"] = ts.dt.hour                  # Line 3: hour feature
    out["trans_dow"] = ts.dt.dayofweek              # Line 4: weekday feature
    out["distance_km"] = compute_distance_km(out)   # Line 5: Haversine
    out["category"] = out["category"].apply(encode) # Line 6: label encode
    return out[FEATURE_COLUMNS + ["Class"]]        # Line 7: select cols
```

| Code line / block | Explanation                                                              |
|-------------------|--------------------------------------------------------------------------|
| Line 1            | Create binary target Class from is_fraud (0=legit, 1=fraud).             |
| Line 2            | Parse string timestamp into datetime for feature extraction.             |
| Line 3            | Hour 0–23 captures time-of-day fraud patterns from EDA.                  |
| Line 4            | Day-of-week 0=Monday captures weekly seasonality.                        |
| Line 5            | Compute km distance cardholder↔merchant — geo fraud signal.              |
| Line 6            | Map category/gender/state strings to integers using training vocabulary. |
| Line 7            | Return only the 13 model features plus target — drop PII columns.        |

## 5. Visualizations — Charts, Graphs & Tables

All figures below were generated from the latest training run on fraudTest hold-out data. Each includes an interpretation paragraph tied to data science decisions.

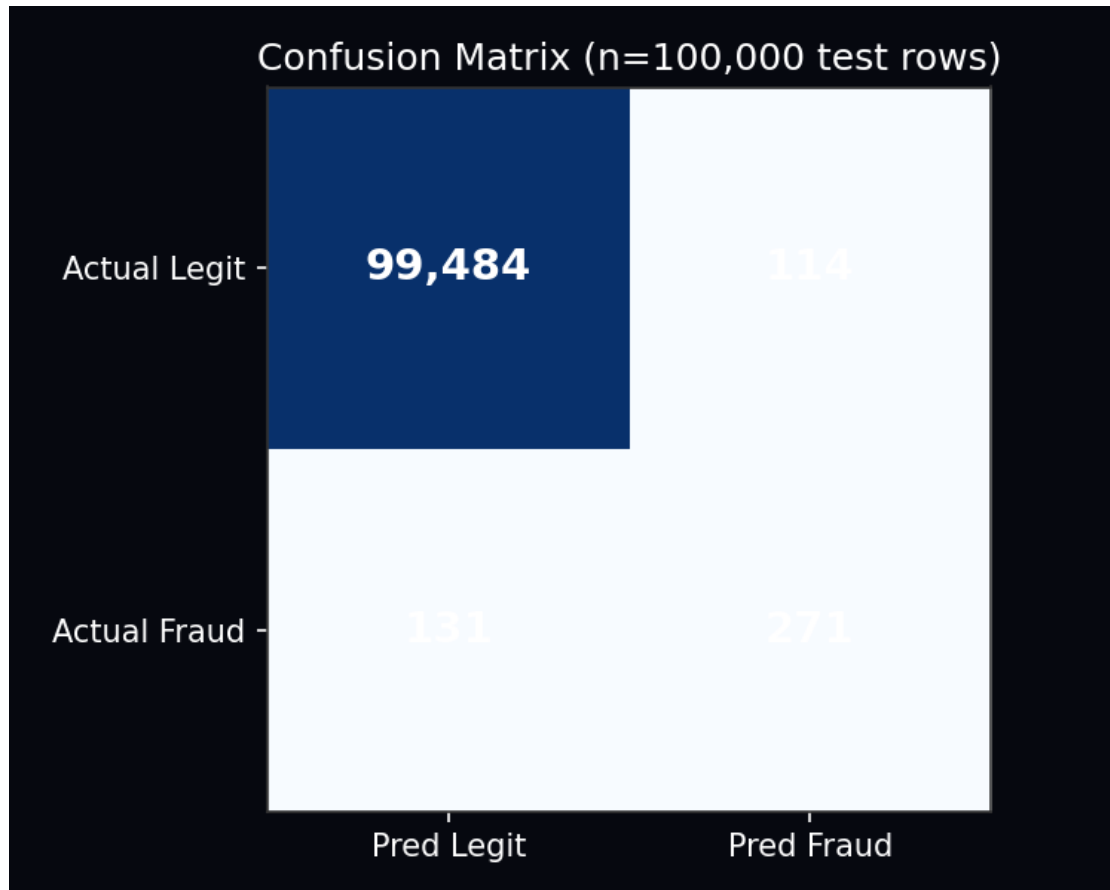
**Figure 7: Summary accuracy metrics**



Interpretation:

- Compares ROC-AUC, Average Precision, F1, Precision, Recall, Balanced Accuracy.
- ROC-AUC 0.985 indicates excellent ranking ability despite imbalance.
- Balanced accuracy (0.836) is more informative than raw accuracy (0.998) for this dataset.

**Figure 8: Confusion matrix (threshold=0.5)**

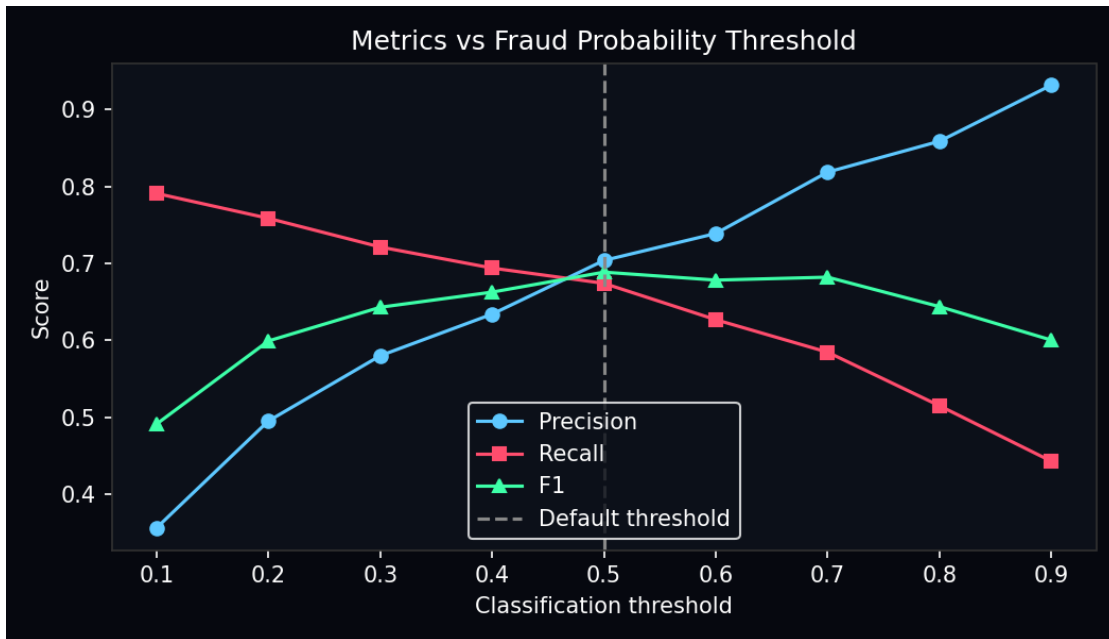


Interpretation:

- True Negatives=99,484: legitimate correctly passed.
- False Positives=114: legitimate incorrectly flagged — customer friction.
- False Negatives=131: fraud missed — financial loss.
- True Positives=271: fraud correctly caught.



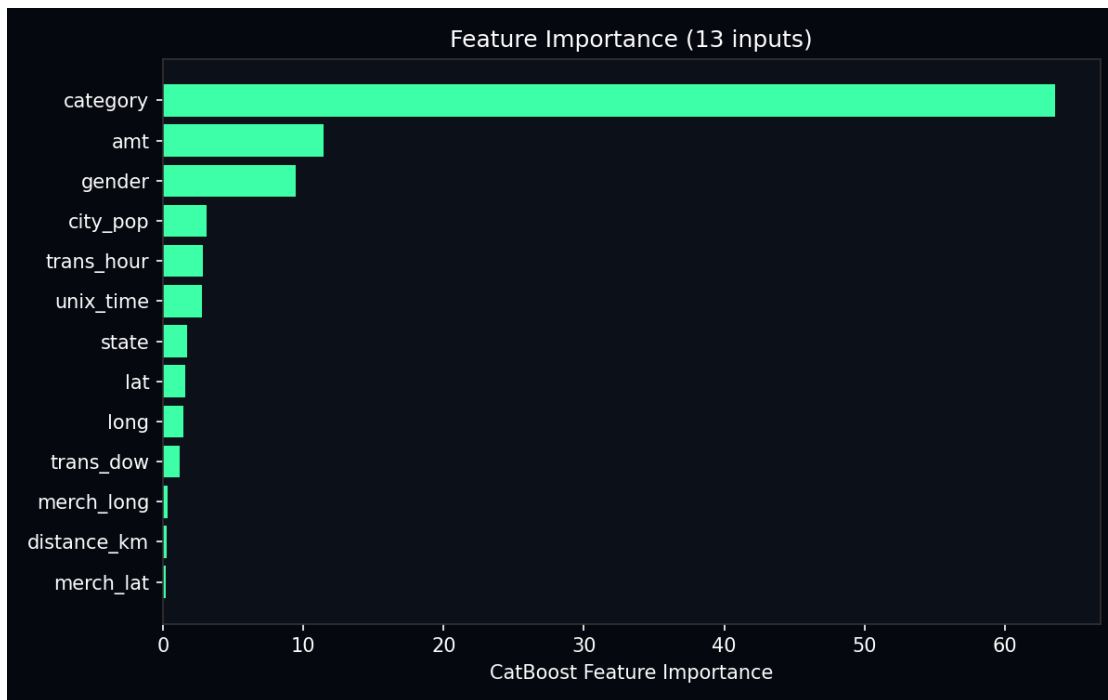
**Figure 9: Threshold sensitivity analysis**



Interpretation:

- Shows precision/recall/F1 trade-off as classification threshold varies 0.1–0.9.
- Lower threshold catches more fraud (higher recall) but increases false alarms.
- Default 0.5 chosen as balanced operating point for coursework demo.

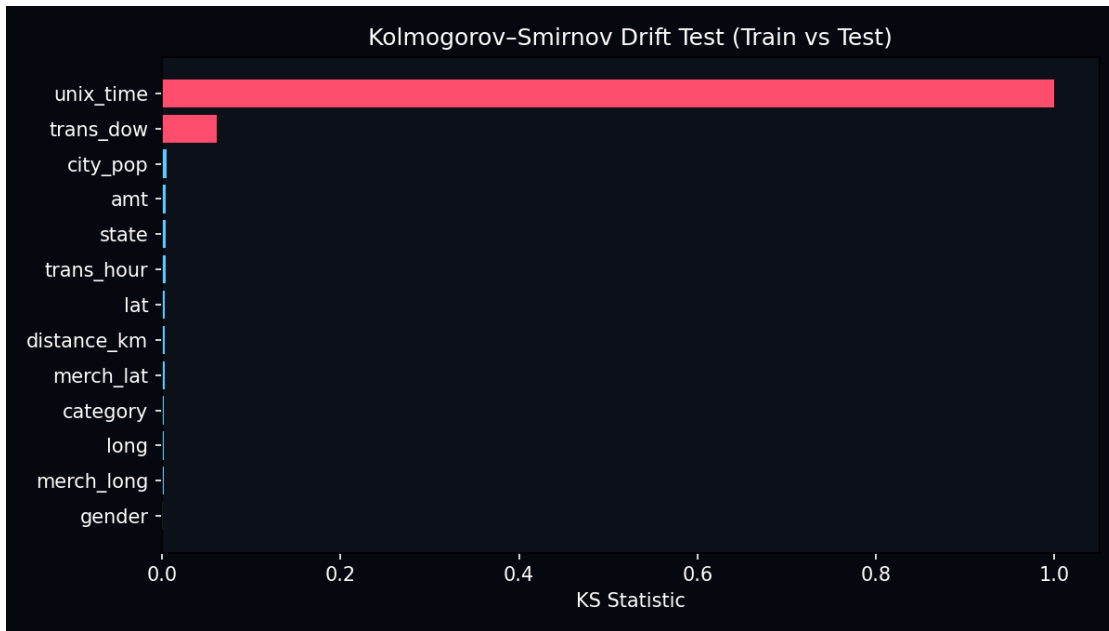
**Figure 10: CatBoost feature importance**



Interpretation:

- Gain-based importance across 500 trees — category dominates at ~64%.
- Confirms EDA finding that merchant type is the strongest fraud signal.
- Geographic features (lat, long, distance\_km) contribute smaller but non-zero gain.

**Figure 11: KS drift statistics (train vs test)**



Interpretation:

- Red bars: features with statistically significant drift ( $p < 0.05$ ).
- `unix_time` KS=1.0 — complete temporal distribution shift between splits.
- Informs governance alerts and explains why test-set scoring may differ from train expectations.

## 5.1 Tabular Outputs

Table 1: Feature importance ranking

| feature     | importance         |
|-------------|--------------------|
| category    | 63.60976712683712  |
| amt         | 11.47331556568562  |
| gender      | 9.458028834214954  |
| city_pop    | 3.0733510150363568 |
| trans_hour  | 2.8253365759437874 |
| unix_time   | 2.7962355426448946 |
| state       | 1.740665115527161  |
| lat         | 1.5552342896744635 |
| long        | 1.4836565313151917 |
| trans_dow   | 1.183353698121395  |
| merch_long  | 0.3200384630754156 |
| distance_km | 0.257810525496898  |
| merch_lat   | 0.2232067164267547 |

Table 2: KS drift report

| feature     | ks_statistic         | p_value                | drift_detected |
|-------------|----------------------|------------------------|----------------|
| unix_time   | 1.0                  | 0.0                    | True           |
| trans_dow   | 0.06098999999999999  | 5.096326418128745e-216 | True           |
| city_pop    | 0.005009999999999999 | 0.0701625730733133     | False          |
| amt         | 0.004205             | 0.1886024973340859     | False          |
| state       | 0.003744999999999999 | 0.3063578335308146     | False          |
| trans_hour  | 0.00373              | 0.3109233933819292     | False          |
| lat         | 0.00355              | 0.3693536596575814     | False          |
| distance_km | 0.003544999999999999 | 0.3710718114013248     | False          |
| merch_lat   | 0.003319999999999999 | 0.453446483151372      | False          |
| category    | 0.002524999999999999 | 0.7879782839665836     | False          |
| long        | 0.00221              | 0.8998948166234672     | False          |
| merch_long  | 0.00218              | 0.908601340144916      | False          |
| gender      | 0.00048              | 1.0                    | False          |

Table 3: Sample API scoring output format

| Field             | Example | Meaning                                  |
|-------------------|---------|------------------------------------------|
| fraud_probability | 0.0180  | Model P(fraud) from CatBoost sigmoid     |
| fraud_prediction  | 0       | Binary flag ( $\geq 0.5$ threshold)      |
| is_fraud_actual   | 1       | Ground truth from CSV — not model output |
| governance_action | alert   | Drift policy result from L1 layer        |

## 6. Accuracy & Model Performance

All metrics computed on fraudTest.csv hold-out (100,000 rows, 402 fraud, 0.40% fraud rate). Train data was SMOTE-balanced; test data reflects natural imbalance.

| Metric            | Value  | Interpretation                                                |
|-------------------|--------|---------------------------------------------------------------|
| ROC-AUC           | 0.9851 | Probability a random fraud row ranks above a random legit row |
| Average Precision | 0.7093 | Area under precision-recall curve                             |
| F1 Score          | 0.6887 | Harmonic mean of precision and recall at threshold 0.5        |
| Precision         | 0.7039 | 70% of flagged transactions are true fraud                    |
| Recall            | 0.6741 | 67% of all fraud cases detected at threshold 0.5              |
| Accuracy          | 0.9976 | Misleadingly high due to 99.6% majority class                 |
| Balanced Accuracy | 0.8365 | Average per-class recall — fairer metric                      |

### 6.1 Model Structure

| Parameter                | Value                           |
|--------------------------|---------------------------------|
| Algorithm                | CatBoost Gradient Boosted Trees |
| Trees (iterations)       | 500                             |
| Max depth                | 6                               |
| Learning rate            | 0.05                            |
| Features                 | 13                              |
| Train rows (post-SMOTE)  | 396,710                         |
| Classification threshold | 0.50                            |

### 6.2 Classification Report

precision recall f1-score support

|   |      |      |      |       |
|---|------|------|------|-------|
| 0 | 1.00 | 1.00 | 1.00 | 99598 |
| 1 | 0.70 | 0.67 | 0.69 | 402   |

|              |      |      |      |        |
|--------------|------|------|------|--------|
| accuracy     |      |      | 1.00 | 100000 |
| macro avg    | 0.85 | 0.84 | 0.84 | 100000 |
| weighted avg | 1.00 | 1.00 | 1.00 | 100000 |

### 6.3 Threshold Analysis

| Threshold | Precision | Recall | F1    | Flagged |
|-----------|-----------|--------|-------|---------|
| 0.1       | 0.356     | 0.791  | 0.491 | 893     |
| 0.2       | 0.495     | 0.759  | 0.599 | 616     |
| 0.3       | 0.580     | 0.721  | 0.643 | 500     |
| 0.4       | 0.634     | 0.694  | 0.663 | 440     |
| 0.5       | 0.704     | 0.674  | 0.689 | 385     |
| 0.6       | 0.739     | 0.627  | 0.678 | 341     |
| 0.7       | 0.819     | 0.585  | 0.682 | 287     |
| 0.8       | 0.859     | 0.515  | 0.644 | 241     |
| 0.9       | 0.932     | 0.443  | 0.600 | 191     |

## 7. Conclusions

This data science project successfully delivered Sentinel, a full-stack fraud detection platform grounded in rigorous EDA, reproducible ETL, and modern gradient boosting. CatBoost achieved ROC-AUC 0.9851 on a realistically imbalanced test set, demonstrating strong discriminative ability. EDA-driven features, especially merchant category, transaction amount, and temporal fields, were validated by post-hoc feature importance.

### 7.1 Key Findings

- EDA must precede ETL: imbalance and category patterns dictated SMOTE and feature selection.
- Geographic distance (Haversine) adds interpretable fraud signal beyond raw coordinates.
- Ground truth (is\_fraud) and model predictions are distinct — recall ~67% at default threshold.
- KS drift on temporal features confirms train/test are not identically distributed.
- Seven-layer architecture enables separation of ML computation from business governance.

### 7.2 Limitations

- Row caps (200k/100k) for practical training — full 1.3M rows would improve coverage.
- Label encoding for categoricals — no explicit embedding of merchant hierarchy.
- No SHAP per-transaction explanations in current UI.
- API lacks authentication — suitable for local demo, not public production.

### 7.3 Future Work

- SHAP/LIME explanations per transaction in Sentinel UI.
- Automated retraining when governance triggers RETRAIN\_RECOMMENDED.
- Threshold optimisation for business-specific cost matrix (FP vs FN costs).
- Cloud deployment with Docker, authentication, and live payment API integration.